چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۵

Kotlin Flow و برنامه‌نویسی واکنشی — Cold و Hot، StateFlow، operatorها و backpressure



اسماعیل مرادی

@esmaeeil-moradi700

چرا Flow روز پنجم است؟

روز ۴ دربارهٔ یک نتیجه در هر بار بود. اپ‌های واقعی با مقدارهایی **در طول زمان** سروکار دارند — تغییرات دیتابیس، سوکت، موقعیت مکانی، ورودی کاربر. در ۲۰۲۶ یعنی `Flow`:

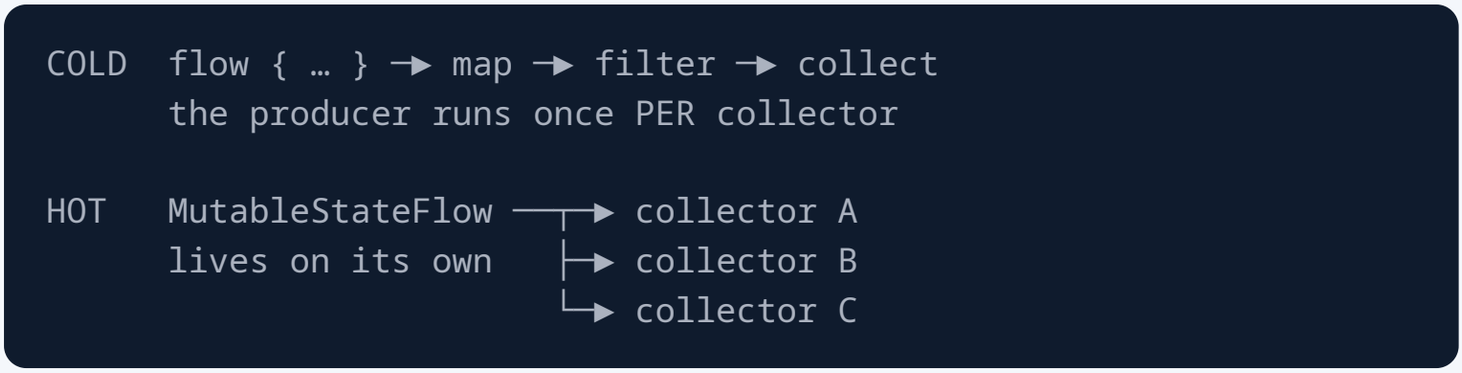
- Room، DataStore و Paging خروجی `Flow` می‌دهند؛ Compose آن را با `collectAsStateWithLifecycle()` جمع می‌کند
- UI state یک `StateFlow` است (روز ۳) — اما انتخاب بین StateFlow، SharedFlow و Channel جایی است که باگ‌ها پنهان می‌شوند
- استراتژی‌های اشتراک (`stateIn` و `shareIn`) تعیین می‌کنند چه چیزی در background به کار ادامه دهد
- backpressure و لغو را coroutineها مجانی می‌دهند — به شرطی که بفهمیدشان
- خیلی از تیم‌ها هنوز در حال مهاجرت از RxJava هستند

سطح	سؤال‌ها
جونیور	مقایسهٔ Flow با List و Sequence، Flowهای cold و collect، operatorها
مید	StateFlow یا SharedFlow یا Channel، stateIn و shareIn، flatten کردن، flowOn
سینیور	backpressure، callbackFlow، collect آگاه از lifecycle
آرکیتکت	خطا و retry، تست با Turbine، مهاجرت از RxJava



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: cold در برابر hot



- **cold:** تا وقتی یک terminal operator مثل `collect`، `first` یا `toList` صدا زده نشود هیچ کاری اجرا نمی‌شود؛ هر collector اجرای مخصوص خودش را می‌گیرد
- **hot:** یک منبع مشترک که بدون collector هم وجود دارد؛ collectorها فقط تماشایش می‌کنند
- **`stateIn`** و **`shareIn`** یک Flow سرد را گرم می‌کنند — و شما تعیین می‌کنید کِی شروع و کِی متوقف شود

سه سؤال برای هر STREAM

چه کسی شروعش می‌کند؟ چه کسی متوقفش می‌کند؟ collectorای که دیر برسد چه می‌بیند؟ به این سه جواب بدهید تا بیشتر باگ‌های Flow ناپدید شوند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

مقایسهٔ Flow با List و Sequence، Flowهای cold و collect، operatorها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. Flow در برابر List و Sequence

	List	Sequence	Flow
ارزیابی	eager — همه با هم	lazy، آیتم به آیتم	lazy، با `collect` اجرا می‌شود
انتظار برای آیتم بعدی	—	ترد را block می‌کند	suspend می‌کند
مقدار در طول زمان (دیتابیس، سوکت)	نه	نه	بله
لغو	—	—	مشارکتی و ساختاریافته

```
interface ArticleRepository {
    suspend fun refresh()                          // one value
    fun observeArticles(): Flow<List<Article>>     // over time
}
```

یک نتیجه یعنی `suspend fun`؛ مقدارها در طول زمان یعنی `Flow`.

تلهٔ مصاحبه

«*Flow را فقط یک بار می‌شود collect کرد.*» — نسخهٔ پارسال این را گفته بود. یک Flow سرد را هر چند بار که بخواهید می‌شود collect کرد؛ هر collect، producer را **از نو اجرا می‌کند**.

تلهٔ مصاحبه

`flow { emit(api.getUser()) }` برای یک فراخوانی یک‌باره — یک `suspend fun` در لباس مبدل: صدا زدنش سخت‌تر، تستش سخت‌تر.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. Flowهای cold و terminal operatorها

```
val ticks: Flow<Int> = flow {
    println("start")            // runs for EVERY collector
    repeat(3) { i ->
        delay(100)
        emit(i)
    }
}

viewModelScope.launch {
    ticks.collect { println(it) }   // start 0 1 2
    ticks.collect { println(it) }   // start 0 1 2 — again
    println("done")                 // after both complete
}
```

- `flow { }` فقط کار را **توصیف می‌کند**؛ terminal operatorها اجرایش می‌کنند: `collect`، `first()`، `toList()`، `single()`، `launchIn(scope)`
- operatorهای میانی (مثل `map` و `filter`) هم lazy هستند
- Flow **ترتیبی** است: `emit` تا وقتی collector مقدار را پردازش نکرده suspend می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چیزهایی که مصاحبه‌گر چک می‌کند

```
lifecycleScope.launch {
    viewModel.state.collect { render(it) }
    viewModel.events.collect { handle(it) }  // ❌ never reached
}
```

تلهٔ مصاحبه

«*خط بعدی به محض شروع `collect` اجرا می‌شود.*» — `collect` تا **تمام شدن** Flow معلق می‌ماند. Room، `StateFlow` و سوکت‌ها هیچ‌وقت تمام نمی‌شوند، پس کد بعد از آن هرگز اجرا نمی‌شود. هر collect بی‌پایان را در coroutine جداگانهٔ خودش launch کنید.

تلهٔ مصاحبه

«*هرگز روی ترد اصلی `collect` نکنید.*» — نسخهٔ پارسال این را گفته بود. collect کردن روی Main کاملاً عادی است: `collect` معلق می‌کند، block نمی‌کند. مهم این است که **کار بالادستی** کجا اجرا می‌شود (سؤال ۷).

سؤال تکمیلی

«*فرق `launchIn(scope)` با `collect` چیست؟*» — `flow.onEach { … }.launchIn(scope)` خلاصهٔ `scope.launch { flow.collect { … } }` است و `Job` را برمی‌گرداند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. operatorهای روزمره

operator	چه می‌کند	کاربرد رایج
`map`، `filter`، `onEach`	تبدیل، حذف، side effect برای هر آیتم	تبدیل DTO به مدل UI، لاگ
`transform`	برای هر آیتم صفر یا چند مقدار emit می‌کند	گسترش یا رد کردن آیتم با منطق
`distinctUntilChanged`	تکراری‌های پشت‌سرهم را رد می‌کند	جلوگیری از کار اضافه
`debounce(ms)`	بعد از یک دورهٔ سکوت emit می‌کند	ورودی جعبهٔ جست‌وجو
`sample(ms)`	آخرین مقدار، یک بار در هر دوره	سنسورهای سریع، پیشرفت کار
`take(n)`، `first()`	بعد از n آیتم متوقف می‌شود	خواندن‌های یک‌باره

تلهٔ مصاحبه

«*با `debounce` یک stream سریع قیمت را نرم کنید.*» — نسخهٔ پارسال این را پیشنهاد داده بود. `debounce` فقط بعد از یک دورهٔ **سکوت** emit می‌کند: اگر به‌روزرسانی‌ها سریع‌تر از timeout برسند، اصلاً چیزی emit نمی‌کند. از `sample` یا `conflate` استفاده کنید.

تلهٔ مصاحبه

«*Flow دو operator به اسم `throttleFirst` و `throttleLatest` دارد.*» — در kotlinx.coroutines نیستند، هرچند نسخهٔ پارسال فهرستشان کرده بود. operatorهای آماده `debounce` و `sample` هستند؛ بقیه را خودتان می‌نویسید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. combine، zip یا merge

```
val uiState = combine(
    repo.cart,                          // Flow<Cart>
    repo.user,                          // Flow<User>
    promo.onStart { emit(null) },       // Flow<String?>
) { cart, user, code ->
    CheckoutUiState(cart, user.address, code)
}
```

	کِی emit می‌کند	کِی تمام می‌شود
`combine`	هر منبعی emit کند — بعد از اینکه همه یک بار emit کرده باشند	همهٔ منبع‌ها تمام شوند
`zip`	همهٔ منبع‌ها آیتم تازه داشته باشند — یک‌به‌یک جفتشان می‌کند	کوتاه‌ترین منبع تمام شود
`merge`	هر منبعی emit کند — هم‌نوع، بدون جفت کردن	همهٔ منبع‌ها تمام شوند

تلهٔ مصاحبه

`combine` صبر می‌کند تا **همهٔ** منبع‌ها یک بار emit کنند. یک منبع ساکت کافی است تا صفحه هیچ چیزی نشان ندهد. به منبع‌های اختیاری یک مقدار شروع بدهید: `onStart { emit(null) }`.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

StateFlow یا SharedFlow یا Channel، stateIn و shareIn،
flatten کردن، flowOn



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. StateFlow، SharedFlow یا Channel

	StateFlow	SharedFlow	Channel
مقدار نگه می‌دارد	همیشه — `.value`	cache اختیاری replay	نه
مقدار اولیه	الزامی	ندارد	ندارد
مقدارهای مساوی	رد می‌شوند (conflate)	تحویل داده می‌شوند	تحویل داده می‌شوند
چند collector	همه آخرین مقدار را می‌بینند	همه هر emission را می‌گیرند	هر آیتم فقط به **یک** گیرنده می‌رسد
وقتی کسی collect نمی‌کند	آخرین مقدار را نگه می‌دارد	دور می‌ریزد (مگر replay یا بافر داشته باشد)	تا ظرفیتش بافر می‌کند، بعد `send` معلق می‌شود
کاربرد	UI state	پخش همگانی برای چند شنونده	صف کار با یک مصرف‌کننده

`StateFlow` و `SharedFlow` **هیچ‌وقت تمام نمی‌شوند** — `collect` روی آن‌ها هرگز برنمی‌گردد. eventهای یک‌بارهٔ UI موضوع روز ۳ بودند: به شکل state مدلشان کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. تله‌های streamهای hot

```
val items = mutableListOf<Item>()
_state.value = items        // emitted
items += newItem
_state.value = items        // same instance: NOT emitted
```

تلهٔ مصاحبه

«*StateFlow همهٔ مقدارها را تحویل می‌دهد.*» — فقط **آخرین** مقدار را نگه می‌دارد، و مقدارهایی را که با مقدار فعلی `equal` باشند رد می‌کند. لیستی که درجا تغییر کند برای collectorها — و برای Compose — نامرئی است. state را تغییرناپذیر نگه دارید.

تلهٔ مصاحبه

«*SharedFlow مقدار اولیه لازم دارد و آخرین مقدار را نگه می‌دارد.*» — این توصیف `StateFlow` است؛ نسخهٔ پارسال هر دو را یکسان توصیف کرده بود.
`MutableSharedFlow()` نه مقدار دارد نه replay، و وقتی کسی collect نمی‌کند emissionها را دور می‌ریزد.

تلهٔ مصاحبه

`tryEmit` روی یک `MutableSharedFlow()` بدون بافر، هر وقت کسی در حال collect باشد `false` برمی‌گرداند — و event بی‌صدا گم می‌شود. یا `extraBufferCapacity` بدهید یا از `emit` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. stateIn و shareIn — از cold به hot

SharingStarted	بالادست کِی شروع می‌شود	بالادست کِی متوقف می‌شود
`Eagerly`	فوراً	هرگز — تا پایان scope
`Lazily`	با اولین collector	هرگز — تا پایان scope
`WhileSubscribed(5_000)`	با اولین collector	۵ ثانیه بعد از رفتن آخرین collector

- خروجی `stateIn` یک `StateFlow` است (مقدار اولیه لازم دارد)؛ خروجی `shareIn` یک `SharedFlow` با `replay` به اندازهٔ N آیتم
- `WhileSubscribed(5_000)`: چرخش صفحه بالادست را از نو شروع نمی‌کند، ولی خروج از اپ متوقفش می‌کند — پیش‌فرض روز ۳ برای UI state

تلهٔ مصاحبه

`Eagerly` در ViewModel: کار قبل از اینکه کسی نگاه کند شروع می‌شود، و وقتی اپ در background است هم ادامه دارد، تا وقتی ViewModel پاک شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. اشتراک یک اتصال

```
class PriceRepository(
    socket: PriceSocket,
    appScope: CoroutineScope,
) {
    // One socket connection, shared by every screen
    val prices: SharedFlow<Price> = socket.prices()
        .shareIn(
            scope = appScope,
            started = SharingStarted.WhileSubscribed(
                stopTimeoutMillis = 5_000,
                replayExpirationMillis = 0,
            ),
            replay = 1,
        )
}
```

`replayExpirationMillis = 0` قیمت cacheشده را همان لحظهٔ توقف سوکت پاک می‌کند — صفحه‌ای که برمی‌گردد هیچ‌وقت قیمت کهنه نشان نمی‌دهد.

تلهٔ مصاحبه

`fun prices() = socket.prices().shareIn(appScope, …)` — هر **فراخوانی** یک shared flow تازه و یک اتصال تازه می‌سازد. یک بار، در یک property، به اشتراک بگذارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. flatten کردن: یک pipeline جست‌وجو

```
val results: StateFlow<SearchState> = query
    .debounce(300)
    .map { it.trim() }
    .distinctUntilChanged()
    .flatMapLatest { q ->
        if (q.isEmpty()) flowOf(SearchState.Idle)
        else repo.search(q)
            .map { SearchState.Hits(it) }
            .onStart<SearchState> { emit(SearchState.Loading) }
            .catch { emit(SearchState.Error) }
    }
    .stateIn(
        viewModelScope,
        SharingStarted.WhileSubscribed(5_000),
        SearchState.Idle,
    )
```

هر query تازه جست‌وجوی در جریان را **لغو می‌کند** — نتیجهٔ کهنه هرگز روی نتیجهٔ تازه نمی‌نشیند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. انتخاب flatMap مناسب

operator	Flowهای داخلی	کاربرد
`flatMapLatest`	مقدار تازه Flow داخلی قبلی را **لغو می‌کند**	جست‌وجو، رفتن از id انتخاب‌شده به جزئیات
`flatMapConcat`	یکی‌یکی، به ترتیب	کار ترتیبی و مرتب
`flatMapMerge`	چندتا هم‌زمان (پیش‌فرض ۱۶)، بدون حفظ ترتیب	کار موازی و مستقل

تلهٔ مصاحبه

`flatMapMerge` برای جست‌وجو: پاسخ کندِ «ko» ممکن است بعد از پاسخ «kotlin» برسد و رویش بنویسد.

تلهٔ مصاحبه

`catch` **بعد از** `flatMapLatest` : اولین خطا کل pipeline را تمام می‌کند — queryهای بعدی دیگر پردازش نمی‌شوند. **داخل** Flow درونی catch کنید، مثل کد بالا.

سؤال تکمیلی

«`mapLatest` *یا* `flatMapLatest` ؟» — `mapLatest` وقتی هر مقدار به **یک** فراخوانی suspend تبدیل می‌شود؛ `flatMapLatest` وقتی به یک stream تبدیل می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. هر operator کجا اجرا می‌شود؟

```
fun logLines(file: File): Flow<LogLine> = flow {
    file.useLines { lines ->
        lines.forEach { emit(it) }      // blocking reads
    }
}
    .flowOn(Dispatchers.IO)             // ↑ runs on IO
    .map { parse(it) }                  // CPU work
    .flowOn(Dispatchers.Default)        // ↑ map runs on Default
```

- `flowOn` context همهٔ چیزهای **بالای** خودش را عوض می‌کند، تا `flowOn` قبلی
- collector جابه‌جا نمی‌شود: `collect { }` همان جایی اجرا می‌شود که صدا زده شده (مثلاً Main)
- عوض شدن dispatcher یک **بافر** بین دو طرف می‌گذارد، تا هم‌زمان اجرا شوند

تلهٔ مصاحبه

`withContext(Dispatchers.IO) { emit(x) }` داخل `flow { }` قاعدهٔ **حفظ context** را می‌شکند و با `IllegalStateException: Flow invariant is violated` شکست می‌خورد. از `flowOn` استفاده کنید — یا از `channelFlow` برای emit از چند coroutine.

تلهٔ مصاحبه

«*`flowOn` collector را به ترد دیگری می‌برد.*» — فقط روی **بالادست** اثر دارد. هر چیزی بعد از آن، از جمله `collect`، در context همان collector می‌ماند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

backpressure، callbackFlow، collect آگاه از lifecycle



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. backpressure: تولیدکنندهٔ سریع، collector کند

به‌طور پیش‌فرض صفی وجود ندارد: `emit` تا وقتی collector مقدار را پردازش نکرده معلق می‌ماند، پس تولیدکننده خودبه‌خود کند می‌شود.

```
val numbers = flow {
    for (i in 1..3) { delay(100); emit(i) }   // 100 ms each
}

numbers.collect { delay(300) }            // ≈ 1.2 s: 1 2 3
numbers.buffer().collect { delay(300) }   // ≈ 1.0 s: 1 2 3
numbers.conflate().collect { delay(300) } // gets 1 3
numbers.collectLatest { delay(300) }      // only 3 finishes
```

- `buffer` اجازه می‌دهد تولیدکننده جلوتر برود — همان مقدارها، در زمان کل کمتر
- `conflate` مقدارهایی را که collector سرش شلوغ بوده و ندیده دور می‌ریزد
- `collectLatest` با رسیدن مقدار تازه، کار نیمه‌تمام collector را لغو می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. انتخاب استراتژی

استراتژی	چه چیزی از دست می‌رود	کاربرد
پیش‌فرض (هیچ)	هیچ — تولیدکننده صبر می‌کند	بیشتر Flowها
`buffer(n)`	هیچ — وقتی پر شود معلق می‌کند	دو مرحلهٔ کند و مستقل
`conflate()`	مقدارهای میانی	قیمت، موقعیت مکانی، پیشرفت کار
`collectLatest { }`	کار نیمه‌تمام collector	رندر سنگین، محاسبهٔ diff
`buffer(n, DROP_OLDEST)`	قدیمی‌ترین مقدارهای صف	تله‌متری، لاگ

تلهٔ مصاحبه

«*`buffer()` می‌تواند تا `OutOfMemoryError` بزرگ شود.*» — نسخهٔ پارسال این را گفته بود. بافر پیش‌فرض محدود است (۶۴ آیتم) و وقتی پر شود تولیدکننده را معلق می‌کند؛ فقط `Channel.UNLIMITED` بی‌حد رشد می‌کند.

تلهٔ مصاحبه

«*Flow مثل Reactive Streams مبتنی بر pull است.*» — هیچ `request(n)` وجود ندارد. backpressure در Flow همان **suspend** ساده است: `emit` تا مقدار تحویل داده نشود برنمی‌گردد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. callbackFlow: بسته‌بندی APIهای مبتنی بر listener

```
fun ConnectivityManager.isOnline(): Flow<Boolean> =
    callbackFlow {
        val callback = object : NetworkCallback() {
            override fun onAvailable(network: Network) {
                trySend(true)
            }
            override fun onLost(network: Network) {
                trySend(false)
            }
        }
        trySend(activeNetwork != null)       // initial value
        registerDefaultNetworkCallback(callback)
        awaitClose { unregisterNetworkCallback(callback) }
    }.distinctUntilChanged()
```

- callbackها تابع suspend نیستند، پس `trySend` . channel به‌طور پیش‌فرض ۶۴ آیتم بافر می‌کند؛ با `conflate()` یا `buffer()` تعیین کنید وقتی پر شد چه شود
- `awaitClose { }` Flow را باز نگه می‌دارد و وقتی collect متوقف شد listener را unregister می‌کند
- فقط یک مقدار؟ `suspendCancellableCoroutine` (روز ۴)

تلهٔ مصاحبه

فراموش کردن `awaitClose` : به محض تمام شدن بلوک، Flow با `IllegalStateException` شکست می‌خورد — و listenerای که هرگز unregister نشود نشت می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. collect آگاه از lifecycle

روش collect	وقتی اپ در background است
`lifecycleScope.launch { flow.collect {} }`	❌ تا DESTROYED به collect ادامه می‌دهد
`launchWhenStarted { }` — منسوخ	❌ collector متوقف، ولی بالادست همچنان در حال اجرا
`repeatOnLifecycle(STARTED)` و `flowWithLifecycle`	✅ در STOP لغو، در START دوباره شروع
`collectAsState()` در Compose	⚠️ تا وقتی در composition است collect می‌کند — lifecycle را نادیده می‌گیرد
`collectAsStateWithLifecycle()`	✅ هم Compose، هم lifecycle

تلهٔ مصاحبه

نسخهٔ پارسال `collectAsState` و `launchWhenStarted` را توصیه کرده بود. در ۲۰۲۶: در Compose از `collectAsStateWithLifecycle()` و در Viewها از `repeatOnLifecycle` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. در کد

```
// Compose
val state by viewModel.uiState.collectAsStateWithLifecycle()

// Views (Fragment)
viewLifecycleOwner.lifecycleScope.launch {
    viewLifecycleOwner.repeatOnLifecycle(STARTED) {
        launch { viewModel.uiState.collect(::render) }
        launch { viewModel.banner.collect(::showBanner) }
    }
}
```

- دو Flow یعنی دو بلوک `launch` داخل `repeatOnLifecycle` (سؤال ۲)
- در Fragment از `viewLifecycleOwner` استفاده کنید، نه خود Fragment (روز ۲)

تلهٔ مصاحبه

`repeatOnLifecycle` در هر START دوباره collect را شروع می‌کند. اگر یک Flow **سرد** را collect کنید که شبکه را صدا می‌زند، هر بار برگشتن به اپ درخواست را تکرار می‌کند. به‌جایش آن را از ViewModel با `stateIn(…, WhileSubscribed(5_000), …)` منتشر کنید.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

خطا و retry، تست با Turbine، مهاجرت از RxJava



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. streamهای مقاوم

```
fun livePrices(): Flow<PriceState> = socket.prices()
    .map<Price, PriceState> { PriceState.Live(it) }
    .retryWhen { cause, attempt ->
        val retry = cause is IOException && attempt < 5
        if (retry) {
            emit(PriceState.Reconnecting)
            delay(1_000L shl attempt.toInt())  // 1 s … 16 s
        }
        retry
    }
    .catch { e ->
        if (e !is IOException) throw e
        emit(PriceState.Failed)
    }
```

- `retryWhen` بالادست را دوباره collect می‌کند — اینجا یعنی یک اتصال سوکت تازه — تا ۵ بار با backoff نمایی، و در این مدت حالت «در حال اتصال دوباره» را emit می‌کند
- `catch` شکست نهایی **شبکه** را به مقداری تبدیل می‌کند که UI بتواند نشانش دهد
- هر چیز دیگری باگ است: دوباره throw می‌شود و به crash reporting می‌رسد (روز ۴، سؤال ۱۱)

سؤال تکمیلی

«*retry* یا *retryWhen*؟» — برای حالت‌های ساده `retry(3) { it is IOException }`؛ و وقتی شمارهٔ تلاش را لازم دارید یا می‌خواهید در زمان انتظار چیزی emit کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. قاعده‌های خطا در Flow

- exception، Flow را **تمام می‌کند**. بعد از اجرای `catch` دیگر هیچ مقداری از بالادست نمی‌رسد.
- `catch` فقط exceptionهای **بالادست** را می‌بیند؛ exceptionای که داخل `collect { }` رخ دهد از آن عبور می‌کند.
- **شفافیت exception:** هرگز `emit` را داخل `try/catch` نگذارید — exceptionهای collector را می‌بلعد.

```
// ❌ Swallows exceptions thrown by the collector
flow {
    try { emit(value) } catch (e: Exception) { log(e) }
}
```

kotlinx.coroutines این را تشخیص می‌دهد: `emit` بعدی با پیام «*Flow exception transparency is violated*» شکست می‌خورد.

تلهٔ مصاحبه

«`catch` *را آخر pipeline بگذارید تا stream زنده بماند.*» — `catch` نمی‌تواند Flow تمام‌شده را ادامه دهد. برای streamهای طولانی، منبع را `retry` کنید یا **داخل** `flatMapLatest` catch کنید (سؤال ۶).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. تست Flow با Turbine

```
@Test fun refreshShowsSpinnerThenPosts() = runTest {
    val repo = FakeFeedRepository(posts, delay = 1.seconds)
    val viewModel = FeedViewModel(repo)

    viewModel.uiState.test {
        assertEquals(FeedUiState(), awaitItem())
        viewModel.refresh()
        assertTrue(awaitItem().isRefreshing)
        assertEquals(posts, awaitItem().posts)
    }
}
```

- `test { }` Flow را collect می‌کند: `awaitItem()`، `awaitComplete()`، `awaitError()`، `expectNoEvents()`
- اگر eventی مصرف‌نشده بماند تست fail می‌شود — تست‌ها دقیق می‌مانند
- برای چند Flow هم‌زمان: `turbineScope { }` به‌همراه `testIn(backgroundScope)`

تلهٔ مصاحبه

`delay` را از fake حذف کنید و ممکن است تست fail شود: `StateFlow` قبل از اجرای collector از «در حال refresh» به «تمام‌شده» می‌رسد و conflation حالت spinner را رد می‌کند. fakeها را suspendکننده بسازید — یا فقط state نهایی را assert کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. از RxJava به Flow

RxJava	معادل در Coroutines و Flow
`Single<T>`، `Completable`، `Maybe<T>`	`suspend fun` با خروجی `T`، `Unit` یا `T?`
`Observable<T>`، `Flowable<T>`	`Flow<T>`
`BehaviorSubject`	`MutableStateFlow`
`PublishSubject`	`MutableSharedFlow`
`switchMap`، `concatMap`، `flatMap`	به ترتیب `flatMapLatest`، `flatMapConcat`، `flatMapMerge`
`subscribeOn`	`flowOn`
`observeOn`	context همان collector
`CompositeDisposable`	یک scope، مثلاً `viewModelScope`

تلهٔ مصاحبه

«`flowOn` همان `observeOn` در *Flow است.*» — برعکس: `flowOn` روی **بالادست** اثر دارد، مثل `subscribeOn` (با این تفاوت که جایش مهم است). نقش `observeOn` را contextای بازی می‌کند که در آن collect می‌کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. مهاجرت بدون بازنویسی کامل

```
// kotlinx-coroutines-rx3 adapters, both directions
val prices: Flow<Price> = legacyApi.priceStream().asFlow()

suspend fun loadUser(id: String): User =
    legacyApi.user(id).await()            // Single → suspend

fun cartUpdates(): Observable<Cart> =
    cartRepository.cart.asObservable()    // for Rx callers
```

- **از لبه‌ها به داخل:** فیچرهای جدید با Flow؛ adapterها در مرز؛ لایهٔ data را repository به repository تبدیل کنید
- هر دو طرف را تست‌شده نگه دارید: Turbine برای Flow، `TestObserver` برای Rx
- وقتی آخرین صدازنندهٔ Rx رفت، adapterها را حذف کنید

نظر من

هزینهٔ واقعی RxJava در ۲۰۲۶ کمبود قابلیت نیست — اجرای **دو مدل هم‌روندی** در یک کدبیس است. مهاجرت را به سبک strangler برنامه‌ریزی کنید، نه یک بازنویسی یک‌جا.



اسماعیل مرادی
@esmaeeil-moradi700

روز ۵ — خلاصهٔ یک‌صفحه‌ای

- یک مقدار یعنی `suspend fun`؛ مقدار در طول زمان یعنی `Flow`؛ Flow سرد برای **هر** collector از نو اجرا می‌شود.
- `collect` تا تمام شدن Flow معلق می‌ماند؛ Flow داغ تمام نمی‌شود، پس هر collect را جدا launch کنید.
- `debounce` برای ورودی، `sample` یا `conflate` برای streamهای سریع؛ `combine` منتظر همهٔ منبع‌هاست.
- `StateFlow` برای state؛ `SharedFlow` برای پخش همگانی؛ `Channel` برای یک مصرف‌کننده.
- با `stateIn` یا `shareIn` در یک property به اشتراک بگذارید — برای UI از `WhileSubscribed(5_000)` استفاده کنید.
- `flatMapLatest` برای جست‌وجو؛ context را با `flowOn` عوض کنید، نه با `withContext` دور `emit`.
- backpressure یعنی suspend: `buffer`، `conflate`، `collectLatest` — بافر پیش‌فرض محدود است.
- `callbackFlow` با `trySend` و `awaitClose`؛ در UI با `collectAsStateWithLifecycle` collect کنید.
- خطا Flow را تمام می‌کند: `retryWhen` با backoff؛ `catch` فقط بالادست را می‌گیرد.

روز ۶ در راه است: ملزومات Jetpack — ViewModel، Room، DataStore و Paging. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700